

# Optimizing LLM Queries in Relational Data Analytics Workloads

SHU LIU\*, UC Berkeley, USA

ASIM BISWAL\*, UC Berkeley, USA

AMOG KAMSETTY, UC Berkeley, USA

AUDREY CHENG, UC Berkeley, USA

LUIS GASPAR SCHROEDER, UC Berkeley, USA and Technical University of Munich, Germany

LIANA PATEL, Stanford University, USA

SHIYI CAO, UC Berkeley, USA

XIANGXI MO, UC Berkeley, USA

ION STOICA, UC Berkeley, USA

JOSEPH E. GONZALEZ, UC Berkeley, USA

MATEI ZAHARIA, UC Berkeley, USA

Batch data analytics is a growing application for Large Language Models (LLMs). LLMs enable users to perform a wide range of natural language tasks, such as classification, entity extraction, and translation, over large datasets. However, LLM inference is highly costly and slow: for example, an NVIDIA L4 GPU running Llama3-8B can only process 6 KB of text per second, taking about a day to handle 15 GB of data; processing a similar amount of data costs around \$10K on OpenAI's GPT-4o. In this paper, we propose novel techniques that can significantly reduce the cost of LLM calls for relational data analytics workloads. Our key contribution is developing efficient algorithms for reordering the rows and the fields within each row of an input table to maximize key-value (KV) cache reuse when performing LLM serving. As such, our approach can be easily applied to existing analytics systems and serving platforms. Our evaluation shows that our solution can yield up to 3.4× improvement in job completion time on a benchmark of diverse LLM-based queries using Llama 3 models. Our solution also achieves a 32% cost savings under OpenAI and Anthropic pricing models.

## ACM Reference Format:

Shu Liu, Asim Biswal, Amog Kamsetty, Audrey Cheng, Luis Gaspar Schroeder, Liana Patel, Shiyi Cao, Xiangxi Mo, Ion Stoica, Joseph E. Gonzalez, and Matei Zaharia. 2026. Optimizing LLM Queries in Relational Data Analytics Workloads. In *Proceedings of Proceedings of Machine Learning and Systems (MLSys'25)*. ACM, New York, NY, USA, 17 pages. <https://doi.org/10.48550/arXiv.2403.05821>

## 1 INTRODUCTION

We start by situating our work in a growing trend: LLMs are no longer just standalone chat tools — we're seeing them get embedded directly into relational analytics pipelines. Major database vendors like AWS Redshift [1], Databricks [7], and Google BigQuery [10] have added LLM invocation directly into their SQL APIs, and DataFrame libraries and frameworks [5, 32] have followed suit. This lets us write queries where an LLM is called once per row of a table — for example, to check whether a customer support response actually addressed a user's request — and we expect this kind

\*Equal contribution.

Authors' Contact Information: Shu Liu, [lsu@berkeley.edu](mailto:lsu@berkeley.edu), UC Berkeley, USA; Asim Biswal, UC Berkeley, USA; Amog Kamsetty, UC Berkeley, USA; Audrey Cheng, UC Berkeley, USA; Luis Gaspar Schroeder, UC Berkeley, USA and Technical University of Munich, Germany; Liana Patel, Stanford University, USA; Shiyi Cao, UC Berkeley, USA; Xiangxi Mo, UC Berkeley, USA; Ion Stoica, UC Berkeley, USA; Joseph E. Gonzalez, UC Berkeley, USA; Matei Zaharia, UC Berkeley, USA.

of usage, spanning classification, entity extraction, summarization, and translation, to keep growing as analysts adopt LLMs for more of their data processing tasks. We refer to this whole class of queries as "LLM queries."

```

SELECT user_id, request,
       support_response,
       LLM('Did {support_response} address
          {request}?') AS success
FROM customer_tickets
WHERE support_response <> NULL

```

The problem we run into is cost and scale. Real-world tables can have millions of rows, and invoking an LLM per row is computationally expensive — we cite the extreme case where a single L4 GPU running Llama3-8B takes roughly a full day to process 15 GB of text, and doing similar work through GPT-4o's API would cost around \$10K.

So we turn to a technique the LLM-serving community has been developing: prompt caching [9, 17, 19, 38, 41], where the attention states (the key-value or "KV" cache) [36] for a shared prompt prefix are stored and reused across requests instead of recomputed from scratch. This doesn't just save compute — providers like OpenAI [30], Anthropic [2], and Google [11] now charge substantially less for cached prompt tokens, so better cache reuse directly cuts our costs too. We observe, however, that just wiring an analytics engine up to an LLM backend that has caching enabled doesn't get us very far, because the requests as generated naturally from a table aren't structured to maximize shared prefixes. Our central insight is that because we're working in a batch setting — the whole table is known ahead of time — we have the freedom to reorder both the rows and the fields within each row before dispatching requests, arranging things so that consecutive requests overlap as much as possible at the start of their prompts.

We frame this as a search problem over an enormous space: for a table with  $n$  rows and  $m$  fields, there are  $n!(m!)^n$  possible orderings. We show that a natural shortcut — applying one fixed field order across the whole table — leaves significant performance on the table, since per-row field reordering can improve our cache hit count by up to a factor of  $m$  over any fixed ordering. This motivates the two algorithms we introduce: **Prefix Hit Recursion (OPHR)**, which finds the provably best ordering but has exponential runtime, and **Greedy Group Recursion (GGR)**, our practical approximation that uses functional dependencies (like primary/foreign key relationships) and table statistics to prune the search space and stay fast even on large tables.

We implement both algorithms on top of Apache Spark [40] with vLLM [19] as the serving backend, and since there wasn't an established benchmark for this kind of workload, we build our own — 16 LLM queries spanning selection, projection, multi-step invocations, and retrieval-augmented generation [21], run across seven real datasets like Amazon Product Reviews [12], Rotten Tomatoes reviews [31], and SQuAD [33]. Across this benchmark, we find our approach delivers 1.5–3.4× speedups in end-to-end query latency and up to 32% cost savings under real OpenAI and Anthropic pricing, all while preserving the original meaning of the queries.

## 2 BACKGROUND AND MOTIVATION

Before we get into our actual solution, we use this section to build up the technical picture that motivates it — essentially, everything a reader needs to understand why *request reordering* is even a meaningful lever to pull.

We begin by walking through how **LLM inference** works at a fairly fundamental level, since our optimization depends on understanding where the real costs come from. Modern LLMs are autoregressive Transformer models [36], meaning they generate output one token at a time, and each new token depends on the entire sequence of tokens that came before it. We break inference into two distinct phases: the prefill phase, where the model processes the full input prompt in one shot, and the decoding phase, where it generates the output tokens sequentially. We note that production serving engines — **vLLM** [19], **TGI** [13], **TensorRT-LLM** [29], and similar systems — try to make this efficient by continuously batching many requests together [39] so the GPU stays busy rather than sitting idle between individual token generations. Even so, we emphasize that inference remains a genuinely expensive operation: all of the intermediate computed state for every token has to be kept in memory as key and value vectors (the **KV cache**), and depending on model size this can add up quickly — we cite a rough estimate of up to 800KB per token for a 13B-parameter model [19], meaning a fairly ordinary 2,000-token request might require over a gigabyte and a half of GPU memory just to process. Combined with the fact that current hardware tops out around 2,000 tokens per second per GPU, we frame this as a real structural bottleneck for anyone trying to run LLMs at the scale that analytical workloads demand — not a minor inefficiency, but a fundamental limit on how much data can realistically be processed and how much it costs.

From there, we shift to explaining prompt KV caching as the mechanism the broader LLM-serving community has developed to blunt this cost, and the mechanism our own work builds on top of. The underlying idea is fairly intuitive: if two different requests happen to share an identical prefix — the same opening sequence of tokens — then the computation and cached internal state from processing that prefix the first time can simply be reused the second time, rather than recomputed from scratch [41]. This means only the portion of the request that’s actually new needs fresh computation, which can meaningfully cut both latency and, since major providers now price cached tokens at a discount, monetary cost as well.

Having established both why inference is expensive and how caching helps, we then turn to why we think relational data in particular is such a promising place to apply this idea. We point out that real-world databases are rarely random — they’re full of structured repetition. Columnar storage formats like C-Store and Parquet [34] already lean on repeated values for compression, and a whole family of existing database techniques — run-length encoding [20], correlation and functional-dependency analysis [8, 15], database cracking [14], multi-dimensional clustering [6], Delta Lake’s Z-ordering [3] — exist specifically to reorganize data around these repeated values and access patterns. Our observation is that this same underlying repetitiveness, which the database community has spent decades exploiting for other purposes, maps naturally onto the KV-caching opportunity from LLM serving. Because an LLM query in our setting is invoked once per row over a table whose full contents are already known ahead of time — this isn’t a live, unpredictable stream of requests — we’re not stuck with whatever row and field order the table happens to arrive in. We have the freedom to rearrange both the order of rows and the order of fields within each row before we ever send anything to the model, specifically to maximize how much of each request’s beginning overlaps with the one before it. We formalize this goal as maximizing what we call the *prefix hit count*, and this framing is what sets up the concrete reordering algorithms — OPHR and GGR — that we introduce in the sections that follow.

### 3 PROBLEM SETUP

This section introduces the problem setup of maximizing prefix hits in the prompt cache (Sec 3.1) and highlights cases where naive fixed field ordering can result in significantly lower hit rates (Sec 3.2).

### 3.1 Setup Objective

In this section, our goal is to pin down, in precise mathematical terms, exactly what we mean when we talk about "maximizing prefix hits" — because up to this point we've only described the idea informally, and everything we build in the rest of the paper (our **OPHR** and **GGR** algorithms) depends on having a rigorous, unambiguous target to optimize against rather than just an intuition. Let's explore an example query:

```
SELECT LLM("Summarize: ",pr.*)
FROM(
  SELECT review,rating,description
  FROM reviews r JOIN product p ON
    r.asin=p.asin
)AS pr
```

We start by describing the general shape of the LLM operator we're designing around, since our reordering strategy only makes sense in the context of how these operators are actually invoked. We define it as a generic function that takes in a natural-language prompt template along with a set of fields drawn from a table — this could be an explicit list like  $T.a, T.b, T.c$ , or it could be all fields at once via  $T.*$ . We deliberately keep this design simple, partly because it mirrors how LLM operators are already implemented in real analytics systems and SQL-like APIs, and partly because this simplicity is exactly what gives us the freedom we need: since the operator just consumes a set of fields rather than a fixed positional structure, we're free to dynamically reorder which fields go into a request, and in what sequence, without changing what the query actually computes. We illustrate this with a running example — a query that joins a reviews table with a product table and asks the LLM to summarize each row using the review text, rating, and description fields — just to make the abstraction concrete before diving into the formalism.

From there, we lay out our actual optimization objective. Given a table with  $n$  rows and  $m$  fields, we're searching for the best possible ordering of both the rows themselves and the fields within each individual row, with the explicit goal of maximizing how much consecutive requests overlap with each other from the very start of the prompt. To reason about this formally, we represent a candidate ordering as a list of tuples: shuffling the list itself changes the order in which rows are sent to the LLM, while shuffling the values within a given tuple changes the order of fields for that particular row — and importantly, we allow this field order to vary independently from row to row, rather than forcing one fixed layout across the whole table.

We then introduce the central metric that everything in the paper builds on: the *prefix hit count*, or **PHC**. For each row, we compare its field values against the same fields in the immediately preceding row, starting from the first field in the ordering and continuing to check subsequent fields only as long as every value up to that point still matches. A match has to be exact — not a partial overlap or a substring — and the sequence of matches has to be unbroken from the start; the moment we hit a field that doesn't match, the count for that row stops there. Rather than simply counting how many fields matched, we weight each hit by the sum of the squared lengths of the matching values. We do this specifically because token-by-token computation in LLM inference scales roughly quadratically with sequence length, so a longer shared prefix isn't just "slightly better" than a short one — it represents disproportionately more compute that we get to skip, and we want our metric to reflect that real-world cost structure rather than just counting matches naively.

We define the *prefix hit count* (PHC) of  $L$  as the number of consecutive field cell values shared with the previous row starting from the first cell, summing over all  $n$  rows. Each cell value must exactly match the corresponding cell of the previous row (cannot be a substring), and cell values past the first must match consecutively (must be a prefix). Formally, a cell in the list of tuples is denoted as  $L[r][f]$ , indicating the value in tuple  $r$  at position  $f$ . Then, the PHC for a list of tuples  $L$  with  $n$  rows and  $m$  fields is given by:

$$\text{PHC}(L) = \sum_{r=1}^n \text{hit}(L, r) \quad (1)$$

Here, the function  $\text{hit}(L, r)$  represents the *prefix hit count* for a single row  $r$  in  $L$ . For simplicity, we assume that the input list is sorted. For each row  $r$ , the function checks if the value in each field  $f$  matches any previously seen value in the same field of the previous row  $r - 1$ . If all previous fields match, the hit count is the sum of the squares of the lengths of the values in those fields until a mismatch occurs. The squared lengths reflect the quadratic complexity of token processing in LLM inference, where each token computation depends on every preceding token and increases computational cost quadratically with input length.

$$\text{hit}(L, r) = \max_{0 \leq c < m} \begin{cases} \sum_{f=1}^c \text{len}(L[r][f])^2 & \text{if } \forall f \leq c, L[r][f] = L[r-1][f] \\ 0 & \text{otherwise} \end{cases} \quad (2)$$

Finally, we introduce two simplifying assumptions that make the problem more tractable without sacrificing much realism. First, we assume that at least one row's worth of data can fit into the KV cache at any given time, since without that baseline assumption there's no meaningful notion of reuse to begin with. Second, we assume that a cache "hit" requires values to match exactly, ruling out partial or substring matches. We defend this second assumption by pointing out that it's not an artificial constraint we're imposing for convenience — exact-value repetition is already a well-established property that real relational databases are built around and actively optimized for, as seen in techniques like run-length encoding [20] and in columnar storage systems like C-Store and Parquet [34]. We're upfront that these are simplifications rather than a perfectly faithful model of every real-world scenario, but we note that this tradeoff pays off: in our later empirical evaluation (Section 6), we show that despite these simplifying assumptions, the resulting approach still performs well on real, messy, real-world data.

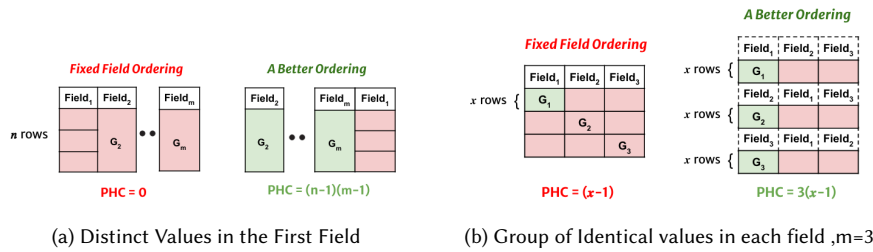


Fig. 1. **Case Study of Fixed Field Ordering:** Comparing the PHC of a fixed field ordering to a better ordering in two scenarios. Green boxes denote cache hits; red boxes indicate cache misses. A box labeled  $G_i$  signifies consecutive rows share the same values in Field  $i$ ; otherwise, assume values are distinct. Fig 1a shows fixed field ordering can be  $(n-1)(m-1)$  worse in terms of PHC compared to an optimized ordering. Fig 1b shows fixed field ordering can be  $m$  times worse in PHC compared to an optimized ordering, where  $m = 3$ .

### 3.2 Case Study: Fixed Field Ordering

In this section, our goal is to make the cost of naive, fixed field ordering concrete rather than just asserting it abstractly. Up to this point we've defined PHC formally, but we haven't yet shown why the way relational data is typically laid out — one fixed column order applied uniformly across every row — is actually a real problem worth solving. So we walk through two deliberately simple, worked examples to build intuition before generalizing.

We start with the most extreme case we can construct: a table with  $n$  rows and  $m$  fields where, for simplicity, every value is exactly one character long. We imagine that the very first field in the table happens to contain completely unique values in every row — something like a timestamp or an ID column, which is extremely common in real data — while every other field repeats the same value across all rows. Because our PHC metric only counts a hit if the match starts from the first field and continues unbroken, this ordering is catastrophic: the moment the first field breaks the streak (which it always does, since every value there is unique), we get zero credit for anything after it, even though the remaining  $m - 1$  fields are all identical from row to row. So this "obvious," default ordering gives us a PHC of exactly 0. But if we simply move that problematic first field to the end instead, keeping the  $m - 1$  repetitive fields up front, every row after the first now hits from the very start, and we get a **PHC** of  $(n - 1)(m - 1)$ . We use this to make a first, blunt point: field order alone, without changing a single value in the table, can be the difference between zero cache reuse and substantial reuse.

We then move to a more realistic and more interesting scenario, since real tables rarely have just one messy field — they tend to have this pattern scattered across several fields at once. We imagine a table where each of the  $m$  fields has its own "clump" of  $x$  consecutive rows sharing an identical value, but these clumps don't line up with each other — they occur in different, non-overlapping stretches of rows depending on which field you look at. We work through a concrete case with  $3x$  rows and 3 fields ( $m = 3$ ). If we insist on one fixed field order applied to the whole table, we can only ever benefit from whichever single field's clump happens to align with our chosen prefix; the other two clumps go completely unused no matter which field we pick, capping our PHC at just  $x - 1$ . But if we're willing to let the field order vary row by row — prioritizing whichever field actually has the matching clump for that particular stretch of rows — we can capture all three clumps instead of just one, tripling our PHC to  $3(x - 1)$ .

We then generalize the intuition from that second example: if this pattern holds across all  $m$  fields, with each field containing roughly the same number of these repeating clumps, then dynamic per-row reordering can yield up to an  $m$ -fold improvement in PHC compared to any single fixed ordering. To make this tangible in dollar terms rather than just an abstract count, we run through a quick calculation under OpenAI's cache-pricing model (which charges half price for cached tokens): for a nine-field table where the naive fixed ordering only achieves a 10% hit rate (e.g.,  $\frac{x-1}{n} = 10$ ), optimizing the field order could realistically translate into 42% in actual cost savings.

We use this section, in short, to justify why we don't settle for a simpler, cheaper-to-compute fixed ordering — the case study shows that doing so leaves real, quantifiable performance and money on the table, which is exactly what motivates the two reordering algorithms (**OPHR** and **GGR**) we introduce next.

## 4 RECURSIVE REQUEST REORDERING

We now introduce our algorithms that re-arrange fields to maximize prefix sharing in the KV cache. We present an optimal recursive reordering algorithm that maximizes PHC (Sec 4.1) and introduce a greedy algorithm that efficiently approximates the optimal algorithm (Sec 4.2).

#### 4.1 Optimal Prefix Hit Recursion (OPHR)

Our Optimal Prefix Hit Maximization (OPHR) algorithm is a recursive algorithm that finds the *optimal* PHC for a given table  $T$  by considering all possible ways to split the table into a group of cells with the same value and two sub-tables. The algorithm takes as input a table  $T$  and computes the optimal PHC  $S$  along with a reordered list of tuples  $L$ . If  $T$  only has one row or field, OPHR computes PHC and trivially returns the sorted  $T$ .

In the recursive case, for each field  $c$  in  $T$ , the algorithm identifies all distinct values  $v$  in the field and the rows  $R_v$  for which the field value is  $v$ . For each distinct value  $v$ , the table is split into two sub-tables: one of  $T$  excluding rows  $R_v$  and one of  $R_v$  excluding field  $c$ . PHC for the currently selected value  $v$  is calculated as the sum of the PHC of the sub-tables and the PHC contribution of  $v$ .

We wrap up our description of OPHR by first noting its practical limitation, then proving it's actually correct. On the practical side, we're upfront that OPHR, despite giving us the best possible PHC, has exponential complexity in both the number of rows and fields. This comes directly from how the algorithm works: at every step it recursively considers every possible grouping of distinct values across every field, and that combinatorial explosion is exactly what makes it impractical for anything but small tables — which is why we introduce a more efficient, greedy alternative later in Section 4.2.

On the correctness side, we walk through a formal proof that OPHR genuinely does find the optimal PHC, and we build this proof using mathematical induction, since the algorithm itself is recursive and induction is the natural tool for showing a recursive procedure behaves correctly at every level of recursion. We start with the base cases, which are deliberately simple so there's no ambiguity about what "optimal" means at the smallest scale. If the table has just a single row, there's no previous row to compare against, so there's fundamentally nothing to reuse — the PHC is trivially 0, and no cleverer algorithm could possibly do better. If instead the table has just a single field, the PHC comes down to summing the squared lengths of each repeated value, but discounting the very first time each value appears, since that first occurrence is a genuine cache miss and can't benefit from anything that came before it. These two cases anchor the proof: they establish that the algorithm behaves correctly at the smallest possible inputs, which is the foundation the inductive argument needs to build on.

For the general, inductive case, we assume as our inductive hypothesis that OPHR already computes the optimal PHC for any table smaller than the one we're currently considering, and our job is to show that this optimality is preserved when we grow the table by one additional row and one additional field. The core mechanism that makes this work is the way OPHR splits the table: for each distinct value appearing in a given field, it divides the table into two sub-tables — one holding all the rows that don't contain that value, and the other holding the rows that do, with that particular field removed from consideration since it's already been accounted for. Because our inductive hypothesis tells us OPHR is already optimal on these two smaller sub-tables, and because the PHC of the full table is exactly equal to the sum of the PHCs of both sub-tables plus whatever contribution came from the value we split on, we've captured the entire picture with nothing left over or double-counted. We emphasize that this splitting step is what makes the whole argument hold together: it keeps each value's contribution to the PHC fully intact rather than letting it get fragmented. If we didn't split the table this way and instead left matching values scattered across non-contiguous groups, we'd risk understating how much reuse is actually achievable. So by consistently splitting along distinct values at every recursive step, OPHR is guaranteed to be considering every genuinely possible configuration, which is exactly why it arrives at the true optimal ordering rather than just a good one.

**Algorithm 1** Greedy Group Recursion (GGR)

---

```

1: Input: Table  $T$ , Functional Dependency  $FD$ 
2: Output: Prefix Hit Count  $S$ , Reordered List of Tuples  $L$ 
3: function HITCOUNT( $v, c, T, FD$ )
4:    $R_v \leftarrow \{i \mid T[i, c] = v\}$ 
5:    $\text{inferred\_cols} \leftarrow \{c' \mid (c, c') \in FD\}$ 
6:    $\text{tot\_len} = \text{len}(v)^2 + \sum_{c' \in \text{inferred\_cols}} \frac{\sum_{r \in R_v} \text{len}(T[r, c'])}{|R_v|}$ 
7:   return  $\text{tot\_len} \times (|R_v| - 1), [c] + \text{inferred\_cols}$ 
8: end function

9: function GGR( $T, FD$ )
10:  if  $|T|_{\text{rows}} = 1$  then
11:    return 0,  $[T[1]]$ 
12:  end if
13:  if  $|T|_{\text{cols}} = 1$  then
14:     $S \leftarrow \sum_{v \in \text{distinct}(T[,1])} \text{HITCOUNT}(v, 1, T)$ 
15:    return  $S, \text{sort}([T[i] \mid i \in 1 \dots |T|_{\text{rows}}])$ 
16:  end if
17:   $\text{max\_HC}, \hat{v}, \hat{c}, \hat{\text{cols}} \leftarrow -1, \text{None}, \text{None}, []$ 
18:  for  $c \in \text{columns}(T), v \in \text{distinct}(T[, c])$  do
19:     $HC, \text{cols} \leftarrow \text{HITCOUNT}(v, c, T, FD)$ 
20:    if  $HC > \text{max\_HC}$  then
21:       $\text{max\_HC}, \hat{v}, \hat{c}, \hat{\text{cols}} = HC, v, c, \text{cols}$ 
22:    end if
23:  end for
24:   $R_{\hat{v}} \leftarrow \{i \mid T[i, \hat{c}] = \hat{v}\}$ 
25:   $A\_HC, L_A \leftarrow \text{GGR}(T[\text{rows} \setminus R_{\hat{v}}, \text{cols}], FD)$ 
26:   $B\_HC, L_B \leftarrow \text{GGR}(T[R_{\hat{v}}, \text{cols} \setminus \hat{\text{cols}}], FD)$ 
27:   $C\_HC, \_ \leftarrow \text{HITCOUNT}(\hat{v}, \hat{c}, T, FD)$ 
28:   $S \leftarrow A\_HC + B\_HC + C\_HC$ 
29:   $L \leftarrow [[\hat{v}] + L_A[i] \mid i \in 1 \dots |R_{\hat{v}}|] + L_B$ 
30:  return  $S, L$ 
31: end function

32: return GGR( $T, FD$ )

```

---

**4.2 Greedy Group Recursion (GGR) Algorithm**

Since OPHR is computationally infeasible at scale, we use this section to introduce Greedy Group Recursion (GGR), our practical approximation, and to justify why it stays fast while still tracking OPHR's quality closely. The GGR algorithm (Algorithm 1) takes a table  $T$  and a set of functional dependencies  $FD$  as input, and returns a prefix hit count  $S$  together with a reordered list of tuples  $L$ . It shares the same base cases as OPHR: if  $T$  has only one row or one column, GGR falls back to the same trivial computation.

Where GGR diverges from OPHR is in the recursive case. Rather than exhaustively considering every distinct value across every field the way OPHR does, GGR greedily selects a single best candidate value  $\hat{v}$  — the one that yields the maximum prefix hit count — at each recursion step (lines 3–8 and 17–23 of the algorithm). It then prioritizes the field  $\hat{c}$  that  $\hat{v}$  belongs to, splits the table into the group of rows sharing that value versus the rest, and recurses on the two

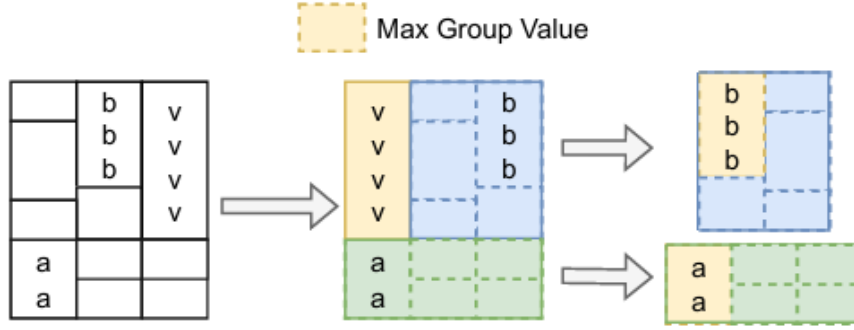


Fig. 2. GGR picks the group with the maximum hit count at each step and calculates PHC as the sum of PHC of the elected group values (yellow box), the sub-table  $T$  excluding rows  $R_v$  (green box), and the sub-table of rows  $R_v$  excluding the field where the value is located in (blue box).

resulting sub-tables (lines 24–26), finally summing the PHC contributions from both sub-tables and from  $\hat{v}$  itself (line 28) – structurally mirroring OPHR’s recursive decomposition, just without OPHR’s exhaustive search.

We explain why this greedy shortcut is so much cheaper: because GGR only ever evaluates one candidate value per step rather than every distinct value in the table, the maximum recursion depth drops to  $O(\min(n, m))$ , since each recursive call strictly shrinks either the row or column dimension of the table. We do note one caveat, though – at each individual recursive step, the cost of scanning the table to identify distinct values can still be quadratic in table size, so the per-step cost isn’t free even though the number of steps is small.

**4.2.1 Functional Dependencies.** We then explain the first of two mechanisms we use to keep GGR both fast and close to optimal: functional dependencies. We define a functional dependency  $FD$  formally as a constraint between two attribute sets  $X$  and  $Y$  in a relation  $R$ , where an instance  $r$  of  $R$  satisfies  $X \leftrightarrow Y$  if, for every pair of tuples  $t_1, t_2 \in r$ :  $t_1.X = t_2.X \implies t_1.Y = t_2.Y$ , and vice versa. Practically, we use these dependencies to narrow down how many fields GGR has to actively consider at each recursive step: once a value  $v$  in field  $f$  is chosen for a row, we automatically place every field functionally dependent on  $f$  immediately alongside it in the final ordering, rather than treating those dependent fields as separate decisions. We illustrate this with a small example: given a table  $R(A, B, C)$  with a functional dependency  $A \leftrightarrow C$ , once field  $A$  has already been placed in the prefix, field  $C$  no longer needs to be considered separately in our recursive search, since it’s already determined.

**4.2.2 Table Statistics.** The second mechanism we introduce is an early-stopping strategy driven by table statistics, intended to cut runtime further beyond what functional dependencies alone achieve. We halt recursion either after a fixed recursion depth (separately for row-wise and column-wise recursion) or once a threshold HITCOUNT score, computed from table statistics, is no longer exceeded. We rely on statistics that are typically already available in databases and analytics systems – cardinality (the number of unique values) and the distribution of value lengths per field. From these, we estimate, for each field  $c$ :

$$\text{HITCOUNT}(c) = \text{avg}(\text{len}(c))^2,$$

which approximates how much that field is likely to contribute to the overall PHC, based on typical value length and frequency. We use this score to prioritize fields more likely to pay off, and once recursion halts early, we fall back to a fixed field ordering for the remaining sub-tables based on these same statistics — letting GGR skip further scanning and recursion on data where it’s unlikely to be worthwhile.

**4.2.3 Achieving Optimal PHC.** Finally, we address the natural question of when our greedy approximation actually matches OPHR’s true optimum rather than just approximating it. We note that GGR trivially matches OPHR whenever the table has only one row or one field, since both algorithms reduce to the same base case. More interestingly, we show that GGR can also reach the true optimum whenever the functional dependencies available are accurate and span all fields in the table — for instance, if a single field  $A$  functionally determines every other field, GGR’s accumulated HITCOUNT score naturally causes it to prioritize groups of values in  $A$  first, capturing the important correlations early and yielding the same reordering OPHR would find. We’re upfront about where this breaks down, though: when multiple fields tie on HITCOUNT, GGR has no mechanism to break the tie the way OPHR’s exhaustive search can, so it may settle for a suboptimal choice in those cases.

## 5 IMPLEMENTATION

In this section, we step back from the algorithms themselves and describe how we actually built and deployed them. We implemented both OPHR and GGR in roughly 1.3 thousand lines of Python, and we chose to evaluate everything on top of PySpark [4], which itself sits on Apache Spark [40] — a large-scale data processing engine we picked specifically because it’s so widely used in industry, which makes our results more representative of how these techniques would actually get adopted in practice rather than being tied to some niche or custom-built system. We explain that the actual LLM invocation is handled by what we call the LLM operator, which we implement as a user-defined function (UDF) inside PySpark — this is the piece that calls out to a configurable LLM endpoint to run inference. This operator takes in a system prompt, a query prompt, and a single row of table data as its inputs, and critically, the order in which the row’s fields and the row itself get passed in isn’t arbitrary — it’s determined directly by whatever ordering our GGR (or OPHR) reordering function computed beforehand. So the reordering algorithm and the actual LLM-calling operator are tightly coupled: one feeds directly into the other.

## 6 EVALUATION

In this section, we evaluate the effectiveness of our optimizations within a constructed benchmark suite of queries. We aim to answer the following questions:

- (1) How does our request reordering optimization impact query latency and costs across different LLM query types and datasets?
- (2) How does the request reordering algorithm influence LLM accuracy for different models?
- (3) What is our algorithm solver time, and how does that compare to end-to-end query latency?

### 6.1 Evaluation Benchmark

Given the lack of standard benchmarks for LLM queries, we construct a benchmark suite to represent real-world data retrieval and processing tasks (Sec 6.1). We define a range of query types (Sec 6.1) over datasets from various sources to assess the impact of LLMs in relational analytics.

| Dataset  | $n_{\text{rows}}$ | $n_{\text{fields}}$ | $\text{input}_{\text{avg}}$ | $\text{output}_{\text{avg}}$ | Query Type |
|----------|-------------------|---------------------|-----------------------------|------------------------------|------------|
| Movies   | 15000             | 8                   | 276                         | {2, 29, 16, 2}               | T1-T4      |
| Products | 14890             | 8                   | 377                         | {3, 107, 62, 2}              | T1-T4      |
| BIRD     | 14920             | 4                   | 765                         | {2, 43}                      | T1, T2     |
| PDMX     | 10000             | 57                  | 738                         | {2, 72}                      | T1, T2     |
| Beer     | 28479             | 8                   | 156                         | {2, 38}                      | T1, T2     |
| SQuAD    | 22665             | 5                   | 1047                        | 11                           | T5         |
| FEVER    | 19929             | 5                   | 1302                        | 3                            | T5         |

Table 1. Datasets:  $n_{\text{rows}}$  and  $n_{\text{fields}}$  denote the number of rows and fields, respectively.  $\text{input}_{\text{avg}}$  and  $\text{output}_{\text{avg}}$  represent average input and output token lengths. Query Type is detailed in Sec 6.1.2. Since  $\text{input}_{\text{avg}}$  remains consistent across query types, we report a single overall average, while  $\text{output}_{\text{avg}}$  varies, with each bracketed value corresponding to a specific query type.

We build our benchmark on seven widely-used datasets spanning recommendation and NLP tasks — Rotten Tomatoes movie reviews [31], Amazon product reviews [12], BIRD [22], PDMX [24], RateBeer reviews [26], SQuAD [33], and FEVER [35]. These vary a lot in shape: row counts range from 10K to nearly 30K, field counts range from just 4 up to 57, and average input/output token lengths differ substantially, letting us stress-test our reordering approach across very different table structures.

We define 16 queries across five query types meant to mirror real-world SQL usage patterns: LLM filter (like a WHERE clause), LLM projection (like a SELECT), multi-LLM invocation (chained calls), LLM aggregation (feeding LLM outputs into functions like AVG), and RAG (retrieval-augmented generation, pulling in external context before answering). Together these cover the main ways we expect analysts to actually invoke LLMs inside relational queries.

We describe how we actually run these experiments: we measure end-to-end query latency and monetary cost across OpenAI and Anthropic endpoints, and hand-label a subset of filter queries to check accuracy. We test primarily with Llama3-8B [27] on a single L4 GPU, plus Llama3-70B [27] on 8 L4s for filter queries, and compare our GGR-based caching approach against two baselines — no caching at all, and caching without any reordering — since we skip evaluating the optimal OPHR algorithm directly, as it’s too slow to run on realistically sized tables. For RAG queries, we embed contexts with a general-purpose text embedding model [23] and retrieve them with FAISS [16]. To assess whether reordering affects accuracy, we use statistical bootstrapping [37] to obtain a distribution of accuracy measurements across repeated runs.

## 6.2 End to End benchmark results

Overview. We show in Fig 3 and Fig 4 that across all 16 queries — filter, projection, multi-LLM invocation, aggregation, and RAG — run with Llama-3-8B on a single L4 GPU, our approach delivers 1.5 to 3.4× speedup over Cache (Original) and 1.8 to 3.8× over No Cache. We then break down what’s driving these numbers for each query type individually. **LLM filter.** We find that filter queries, which tend to produce short outputs like "Yes/No" or sentiment labels, benefit the most from caching: Cache (GGR) achieves 2.1–3.8× speedup over No Cache, while plain caching without reordering only gets a modest 1.03–1.9× boost. We attribute this gap directly to our reordering — GGR further cuts latency by 1.8–3.0× over Cache (Original) by rearranging rows and fields to maximize how much of each prompt’s prefix can be reused. Most of our review-style datasets — Movies, Products, and BIRD — start out with highly unique fields (like review content) due

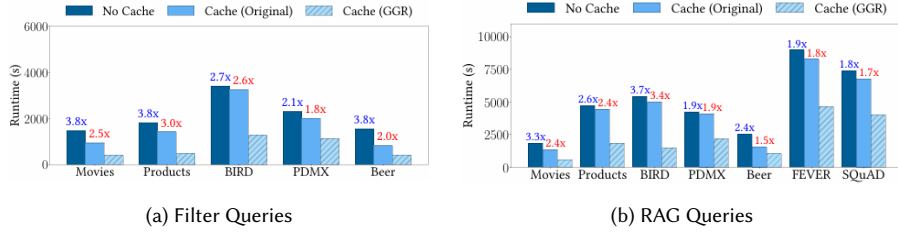


Fig. 3. End-to-end Result (Filter, Selection, RAG): Our optimizations Cache (GGR) achieve 1.5–3.4× speed-up in end-to-end runtime over caching without reordering (Cache (Original)), and 1.8–3.8× over No Cache baseline.

to how they’re joined with metadata tables, which hurts caching by default. We show that reprioritizing fields with more repeated values, like description or product title, raises prefix hit rates by 57–74 % and gives us a 2.5–3× speedup. We highlight PDMX as a harder case, since its 57 fields are mostly long and unique, so even though we boost its hit rate from 12 % to 57 % we only get a 1.8× speedup because 43 % of requests still miss the cache. Beer, by contrast, already has some natural duplication, so GGR pushes its hit rate from 50 % to 80 % for a 2× speedup. **LLM projection.** We test this on longer-output tasks like summarization and find Cache (GGR) achieves 2.4–3.7× speedup over No Cache and 1.5–3.4× over Cache (Original). We note that as output length grows, the decode stage starts to dominate runtime, which shrinks the relative benefit of prefill caching compared to filter queries — except on datasets like BIRD and PDMX, where long strings mean caching also saves meaningful memory during decoding. **Multi-LLM invocation.** We

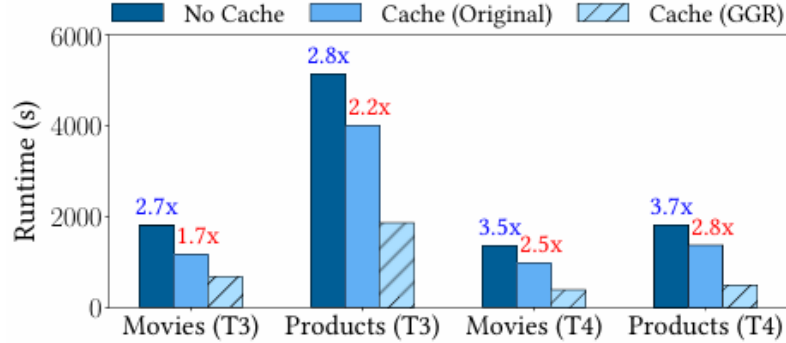


Fig. 4. End-to-end Result (Multi-LLM Invocation, Aggregation): Our optimizations Cache (GGR) achieve 1.7–2.8× speed-up over Cache (Original), and 2.7–3.7× speed-up over No Cache.

combine a filter step with a projection step and find Cache (GGR) achieves 2.7–2.8× speedup over No Cache and 1.7–2.2× over Cache (Original). We explain that this relative gain is smaller than for single-step queries because the first (filter) invocation runs over already-distinct reviews, so caching and reordering don’t help as much there — the benefit mostly comes from wherever the second invocation dominates runtime. **LLM aggregation.** We use an AVG operator over LLM-generated sentiment scores and see 3.5–3.7× speedup over No Cache and 2.5–2.8× over Cache (Original), results we describe as closely mirroring the filter-query numbers since output lengths are similarly short. **RAG.** We evaluate this on FEVER and SQuAD, retrieving supporting contexts via vector search before answering, and find Cache (GGR) achieves 1.9× speedup over No Cache and 1.7–1.8× over Cache (Original). We attribute this to GGR’s ability to rearrange

| Method          | Movies | Prods. | BIRD | PDMX | Beer | FEVER | SQuAD |
|-----------------|--------|--------|------|------|------|-------|-------|
| <b>Original</b> | 35%    | 27%    | 10%  | 12%  | 50%  | 11%   | 11%   |
| <b>GGR</b>      | 86%    | 83%    | 85%  | 57%  | 80%  | 67%   | 70%   |

Table 2. PHR (%) of LLM Filter and RAG queries for Original and GGR, which achieves 30 – 75% higher hit rates.

overlapping contexts across questions, boosting prefix hit rate by 56–59 % over the unsorted baseline. Results on the

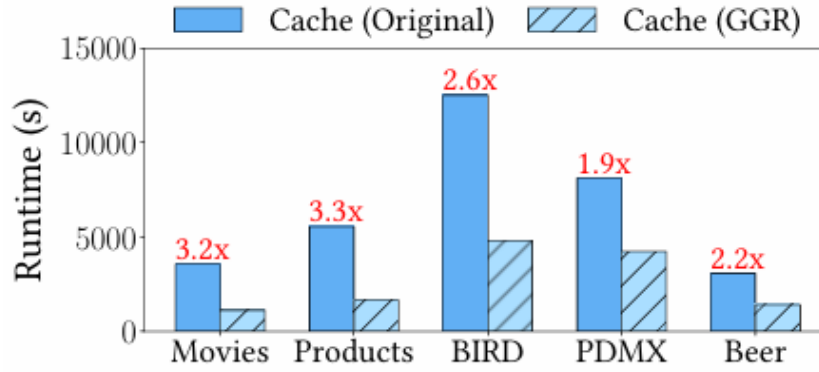


Fig. 5. Cache(GGR) is able to achieve 1.9–3.3× speed-up over Cache(Original) for filter queries on Llama3-70B.

larger Llama-3-70B model, run across 8 L4 GPUs, show a similar pattern — Cache (GGR) achieves 1.9–3.3× speedup over Cache (Original), confirming our approach’s benefits hold as model size scales up.

### 6.3 Cost Savings on Proprietary API Endpoints

We test our GGR reordering against real OpenAI [30] and Anthropic [2] pricing, since both offer discounts for cached tokens. Because both providers require a minimum 1,024-token prefix to cache at all, we duplicate FEVER’s fields to simulate a more realistic long-input dataset. We find GGR delivers 32 % cost savings on GPT-4o-mini and 21 % on Claude 3.5 Sonnet, while the original ordering gets 0 % cache hits since its prefixes never meet the minimum threshold. Extrapolating from our earlier hit-rate measurements across all seven datasets, we estimate GGR could yield 20–39 % savings under OpenAI’s pricing and up to 79 % under Anthropic’s, assuming future support for arbitrary-length prefix caching.

| Dataset | Model   | Method   | PHR (%) | Cost (\$) | Savings (%) |
|---------|---------|----------|---------|-----------|-------------|
| FEVER   | 4o-mini | Original | 0.0     | 0.81      | –           |
|         |         | GGR      | 62.2    | 0.55      | 32%         |
|         | Sonnet  | Original | 0.0     | 5.49      | –           |
|         |         | GGR      | 30.6    | 4.33      | 21%         |

Table 3. OpenAI and Anthropic Costs: cache hit rate (HR%), cost, and savings comparison of GGR over Original for GPT-4o-mini and Claude 3.5 Sonnet in FEVER.

| Dataset  | PHR (%)  |      | Est. Cost Savings (%) |           |
|----------|----------|------|-----------------------|-----------|
|          | Original | GGR  | OpenAI                | Anthropic |
| Movies   | 34.6     | 85.7 | 31                    | 73        |
| Products | 26.7     | 83.3 | 33                    | 73        |
| BIRD     | 10.4     | 84.8 | 39                    | 79        |
| PDMX     | 11.8     | 56.6 | 24                    | 48        |
| Beer     | 49.9     | 80.1 | 20                    | 55        |
| FEVER    | 11.2     | 67.4 | 30                    | 60        |
| SQuAD    | 11.0     | 69.7 | 31                    | 63        |

Table 4. Estimated cost savings: across datasets using PHR from Sec 6.2 and OpenAI and Anthropic’s pricing model.

#### 6.4 Impact of Reorder ingon Accuracy

Since GGR changes the order fields appear in the prompt, we check whether that hurts model accuracy on LLM filter queries and one RAG query (FEVER), using bootstrapped accuracy distributions across 10K runs on Llama-3-8B, Llama-3-70B, and GPT-4o. We find GGR’s accuracy stays within about 5 % of the original ordering in almost every case — the one notable exception is FEVER with Llama-3-8B, where GGR actually improves accuracy by 14.2 % because it happens to place the "claim" field at the end of the prompt, which that particular model prefers. We note this effect doesn’t hold for the larger models, and overall conclude that bigger models like Llama-3-70B and GPT-4o are more robust to field reordering.

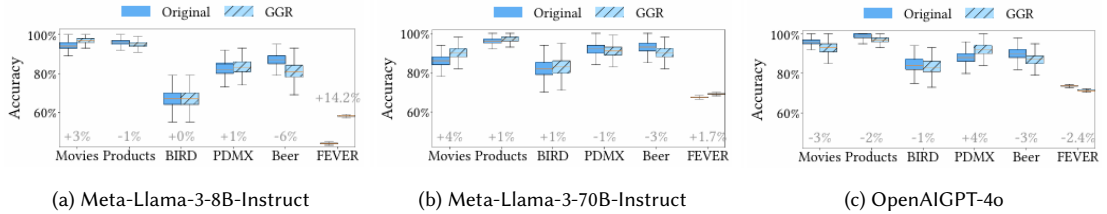


Fig. 6. Accuracy of original v.s. GGR ordering: we perform statistical bootstrapping to get a distribution of exact match accuracy measurements across 10,000 runs. The numbers indicate the difference in the median accuracy of GGR compared to the original ordering..

#### 6.5 Algorithm Overhead

We check whether GGR’s own runtime eats into the speedups it provides, looking separately at latency and memory. On latency, we find GGR consistently finishes in under 15 seconds across all our datasets — even ones with up to 30K rows and 57 fields — which works out to less than 0.01 % of total LLM query runtime, so the reordering overhead is effectively negligible. On memory, we note that GGR only ever needs the specific table touched by the query loaded into

memory, and since its recursive splitting keeps shrinking the table at each step, total memory usage stays at  $O(n \times m)$ , with just minor additional overhead from the recursion stack and temporary variables.

## 7 RELATED WORK

Our optimizations build on recent work in LLM inference as well as prior work integrating machine learning and data management.

**Inference-optimized systems.** There has been a recent rise of dedicated systems for LLM inference, including FasterTransformer [28], Orca [39], vLLM [19], and SGLang [41]. Many of these systems already explore memory-efficient GPU kernels that perform inference while leveraging shared prefixes — SGLang’s RadixAttention [41], Hydragen [17], and Cascade Inference [38] all implement optimized kernels along these lines. Our work builds upon this prior work on high-throughput LLM inference and prefix caching for model serving, and additionally leverages full workload information from batch queries to further improve performance in relational workloads.

**LLMs in relational data analytics.** Many systems support calling LLMs as operators on relational data, spanning from production database vendors like Databricks [7], Google BigQuery [10], and AWS Redshift [1] to programming frameworks like LOTUS [32]. While these works provide APIs for running LLMs over relational data, they do not explore how reordering data can optimize KV cache hits. There is also a line of work [18, 25] that explores using cheaper models for approximate query generation; this orthogonal direction is not considered in our paper’s scope, as our work specifically focuses on calling LLMs as functions from inside a regular, given SQL query.

## 8 CONCLUSION

In this paper, we introduce techniques to optimize LLM in vocations in relational data analytics workloads. By leveraging workload information coupled with observations about the LLM inference process, we can significantly improve end-to-end query performance and reduce costs without affecting query semantics. Our technique achieves upto  $3.4\times$  decreases in end-to-end query latency with Llama-3-8B and Llama-3-70B and also achieves upto 32% cost savings under OpenAI and Anthropic pricing models.

## Acknowledgments

We thank Soujanya Ponnappalli for helpful discussions and feedback, and Jelani Nelson for academic advising. This research was supported by gifts from Accenture, AMD, Anyscale, Broadcom Inc., Google, IBM, Intel, Intesa San Paolo, Lambda, Mibura Inc., Samsung SDS, and SAP.

## References

- [1] Amazon Web Services. [n. d.]. Large Language Models for Sentiment Analysis with Amazon Redshift ML (Preview). <https://aws.amazon.com/blogs/big-data/large-language-models-for-sentiment-analysis-with-amazon-redshift-ml-preview/>. Accessed 01-03-2024.
- [2] Anthropic. 2024. Prompt Caching with Claude. <https://www.anthropic.com/news/prompt-caching>.
- [3] Michael Armbrust, Tathagata Das, Liwen Sun, Burak Yavuz, Shengsheng Zhu, Mukul Murthy, Joseph Torres, Herman van Hovell, Adrian Ionescu, Alicja Luszczak, et al. 2020. Delta Lake: High-Performance ACID Table Storage over Cloud Object Stores. *Proceedings of the VLDB Endowment* 13, 12 (2020), 3411–3424.
- [4] Michael Armbrust, Reynold S. Xin, Cheng Lian, Yin Huai, Davies Liu, Joseph K. Bradley, Xiangrui Meng, Tomer Kaftan, Michael J. Franklin, Ali Ghodsi, and Matei Zaharia. 2015. Spark SQL: Relational Data Processing in Spark. In *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data (SIGMOD ’15)*. Association for Computing Machinery, New York, NY, USA, 1383–1394. doi:10.1145/2723372.2742797
- [5] Harrison Chase. 2022. LangChain. <https://github.com/langchain-ai/langchain>.

- [6] Tao Chen, Nevin L. Zhang, Tengfei Liu, Kin Man Poon, and Yi Wang. 2012. Model-Based Multidimensional Clustering of Categorical Data. *Artificial Intelligence* 176, 1 (2012), 2246–2269.
- [7] Databricks. [n. d.]. AI Functions on Databricks. <https://docs.databricks.com/en/large-language-models/ai-functions.html>. Accessed 01-03-2024.
- [8] Sašo Džeroski. 2003. Multi-Relational Data Mining: An Introduction. *ACM SIGKDD Explorations Newsletter* 5, 1 (2003), 1–16.
- [9] In Gim, Guojun Chen, Seung-seob Lee, Nikhil Sarda, Anurag Khandelwal, and Lin Zhong. 2024. Prompt Cache: Modular Attention Reuse for Low-Latency Inference. <https://arxiv.org/abs/2311.04934>.
- [10] Google Cloud. [n. d.]. LLM with Vertex AI Only Using SQL Queries in BigQuery. <https://cloud.google.com/blog/products/ai-machine-learning/llm-with-vertex-ai-only-using-sql-queries-in-bigquery>. Accessed 01-03-2024.
- [11] Google DeepMind. 2024. Context Caching – Gemini API. <https://ai.google.dev/gemini-api/docs/caching?lang=python>.
- [12] Ruining He and Julian McAuley. 2016. Ups and Downs: Modeling the Visual Evolution of Fashion Trends with One-Class Collaborative Filtering. In *Proceedings of the 25th International Conference on World Wide Web (WWW '16)*. International World Wide Web Conferences Steering Committee, Republic and Canton of Geneva, CHE, 507–517. doi:10.1145/2872427.2883037
- [13] Hugging Face. 2023. Text Generation Inference. <https://huggingface.co/docs/text-generation-inference/en/index>.
- [14] Stratos Idreos, Martin L. Kersten, Stefan Manegold, et al. 2007. Database Cracking. In *CIDR*, Vol. 7. 68–78.
- [15] Ihab F. Ilyas, Volker Markl, Peter Haas, Paul Brown, and Ashraf Aboulnaga. 2004. CORDS: Automatic Discovery of Correlations and Soft Functional Dependencies. In *Proceedings of the 2004 ACM SIGMOD International Conference on Management of Data*. 647–658.
- [16] Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2019. Billion-Scale Similarity Search with GPUs. *IEEE Transactions on Big Data* 7, 3 (2019), 535–547.
- [17] Jordan Juravsky, Bradley Brown, Ryan Ehrlich, Daniel Y. Fu, Christopher Ré, and Azalia Mirhoseini. 2024. Hydragen: High-Throughput LLM Inference with Shared Prefixes.
- [18] Daniel Kang, John Emmons, Firas Abuzaid, Peter Bailis, and Matei Zaharia. 2017. NoScope: Optimizing Neural Network Queries over Video at Scale. *Proc. VLDB Endow.* 10, 11, 1586–1597.
- [19] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP '23)*. Association for Computing Machinery, New York, NY, USA, 611–626. doi:10.1145/3600006.3613165
- [20] Daniel Lemire and Owen Kaser. 2011. Reordering Columns for Smaller Indexes. *Information Sciences* 181, 12 (2011), 2550–2570.
- [21] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2021. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.
- [22] Jinyang Li, Binyuan Hui, Ge Qu, Jiayi Yang, Binhua Li, Bowen Li, Bailin Wang, Bowen Qin, Ruiying Geng, Nan Huo, et al. 2024. Can LLM Already Serve as a Database Interface? A Big Bench for Large-Scale Database Grounded Text-to-SQLs. *Advances in Neural Information Processing Systems* 36 (2024).
- [23] Zehan Li, Xin Zhang, Yanzhao Zhang, Dingkun Long, Pengjun Xie, and Meishan Zhang. 2023. Towards General Text Embeddings with Multi-Stage Contrastive Learning. *arXiv preprint arXiv:2308.03281* (2023).
- [24] Philip Long, Zachary Novack, Taylor Berg-Kirkpatrick, and Julian McAuley. 2024. PDMX: A Large-Scale Public Domain MusicXML Dataset for Symbolic Music Processing. <https://arxiv.org/abs/2409.10831>.
- [25] Yao Lu, Aakanksha Chowdhery, Srikanth Kandula, and Surajit Chaudhuri. 2018. Accelerating Machine Learning Inference with Probabilistic Predicates. In *Proceedings of the 2018 International Conference on Management of Data (SIGMOD '18)*. Association for Computing Machinery, New York, NY, USA, 1493–1508.
- [26] Julian McAuley, Jure Leskovec, and Dan Jurafsky. 2012. Learning Attitudes and Attributes from Multi-Aspect Reviews. <https://arxiv.org/abs/1210.3926>.
- [27] Meta AI. 2024. Meta Llama 3. <https://ai.meta.com/blog/meta-llama-3/>.
- [28] NVIDIA. 2023. FasterTransformer. <https://github.com/NVIDIA/FasterTransformer>.
- [29] NVIDIA. 2023. TensorRT-LLM. <https://github.com/NVIDIA/TensorRT-LLM>.
- [30] OpenAI. [n. d.]. Pricing. <https://openai.com/pricing>. Accessed 01-03-2024.
- [31] Bo Pang and Lillian Lee. 2005. Seeing Stars: Exploiting Class Relationships for Sentiment Categorization with Respect to Rating Scales. In *Proceedings of the ACL*.
- [32] Liana Patel, Siddharth Jha, Carlos Guestrin, and Matei Zaharia. 2024. LOTUS: Enabling Semantic Queries with LLMs over Tables of Unstructured and Structured Data. <https://arxiv.org/abs/2407.11418>.
- [33] Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. 2016. SQuAD: 100,000+ Questions for Machine Comprehension of Text.
- [34] Michael Stonebraker, Daniel J. Abadi, Adam Batkin, Xuedong Chen, Mitch Cherniack, Miguel Ferreira, Edmond Lau, Amerson Lin, Sam Madden, Elizabeth O’Neil, et al. 2018. C-Store: A Column-Oriented DBMS. In *Making Databases Work: The Pragmatic Wisdom of Michael Stonebraker*. 491–518.
- [35] James Thorne, Andreas Vlachos, Christos Christodoulopoulos, and Arpit Mittal. 2018. FEVER: A Large-Scale Dataset for Fact Extraction and Verification. <https://arxiv.org/abs/1803.05355>.
- [36] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. 2023. Attention Is All You Need.
- [37] Rand R. Wilcox. 2003. Bootstrap Confidence Interval. In *Applying Contemporary Statistical Techniques*. Academic Press.

- [38] Zihao Ye, Rulin Lai, Bin-Rui Lu, Chien-Yu Lin, Siyuan Zheng, Lequn Chen, Tianqi Chen, and Luis Ceze. 2024. Cascade Inference: Memory Bandwidth Efficient Shared Prefix Batch Decoding. <https://flashinfer.ai/2024/02/02/cascade-inference.html>.
- [39] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. 2022. Orca: A Distributed Serving System for Transformer-Based Generative Models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. USENIX Association, Carlsbad, CA, 521–538.
- [40] Matei Zaharia, Mosharaf Chowdhury, Tathagata Das, Ankur Dave, Justin Ma, Murphy McCauly, Michael J. Franklin, Scott Shenker, and Ion Stoica. 2012. Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing. In *9th USENIX Symposium on Networked Systems Design and Implementation (NSDI 12)*. 15–28.
- [41] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Jeff Huang, Chuyue Sun, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. 2023. Efficiently Programming Large Language Models Using SGLang.